D3 - Hybrid Encryption

Team 1:
Gonzalez Fernandez Jonathan Uriel – 420051871 / jonathanuri123@gmail.com
Gabriel Hernández Yukioayax Canek – 321058526 / yukio.gabriel.h@gmail.com
Paz Rosas Jesús Eduardo – 315134854 / jesuspazrosas@gmail.com
Vázquez Apolonio Armando – 315140725 / armandovaz090@gmail.com

25 oct 2026

1. Hybrid Design Explanation

Why hybrid encryption is used

Pure asymmetric cryptography (such as ECIES over secp256k1) relies on computationally intensive mathematical operations, including point multiplications and scalar arithmetic over elliptic curves. These operations introduce significant processing overhead, making asymmetric algorithms inefficient and impractical for encrypting arbitrary bulk payloads or high-throughput streams. Additionally, asymmetric primitives enforce strict payload size boundaries dictated by their key lengths.

Conversely, symmetric encryption provides optimal throughput at line rate by taking full advantage of modern hardware optimizations (such as Intel AES-NI and ARM Cryptography extensions). However, symmetric schemes suffer from the fundamental key distribution problem: both communicating parties must establish a shared secret across an already secure channel.

Hybrid encryption resolves this operational trade-off: bulk data is encrypted using high-performance symmetric cryptography under a freshly generated, single-use Data Encryption Key (DEK). Asymmetric cryptography is then used strictly to encapsulate and distribute this lightweight DEK to authorized entities, eliminating the need for pre-shared secrets while maintaining maximum processing throughput.

Why symmetric encryption is still needed

Symmetric encryption—specifically AES-256 in Galois/Counter Mode (GCM)—remains irreplaceable within the architecture for three key cryptographic reasons:

- **Throughput and Scalability:** AES-GCM runs in hardware pipelines, allowing it to encrypt multi-megabyte and gigabyte payloads with negligible CPU overhead and predictable memory consumption.
- **Payload Independence:** Asymmetric algorithms cannot ingest arbitrary file sizes directly; AES-GCM can encrypt arbitrary data streams up to 64 GB per key/nonce pair without size degradation.

- **Simultaneous Authenticity (AEAD):** AES-GCM pairs counter-mode confidentiality with a polynomial MAC (GHASH) to yield a 128-bit authentication tag, detecting in-flight ciphertext corruption before buffer consumption.

Why per-recipient key encryption is required

In a multi-user document vault, access control must remain flexible, granular, and cryptographically isolated. Re-encrypting the payload file separately for each recipient would result in prohibitive $O(N)$ storage expansion and transmission bandwidth. The D3 architecture solves this through per-recipient encapsulation:

- **Single Payload Encryption:** The document is encrypted exactly once using a cryptographically random, ephemeral 256-bit DEK.
- **Independent Asymmetric Encapsulation:** The 32-byte DEK is wrapped independently for each recipient using ECIES with their respective secp256k1 public key, generating a dedicated encrypted_key entry.
- **Key Isolation & Forward Secrecy:** The compromise of one recipient's private key only compromises the DEK for that specific vault container, providing zero cryptographic access to the private keys or access rights of peer recipients.
- **Efficient Lifecycle Management:** Adding or revoking a participant only requires appending or removing a lightweight metadata entry from the recipient table, rather than decrypting and re-encrypting the entire bulk dataset.

2. Security Decisions

How do recipients identify their key?

To prevent "blind trial decryption" (where a user must attempt costly asymmetric operations on every entry until one works), D3 implements a two-tiered deterministic identification model:

- **User Identifier (id):** A validated alphanumeric string (1–64 characters matching ^[A-Za-z0-9_-]{1,64}$) uniquely representing the recipient entity. The decryption engine performs an immediate lookup to isolate the target recipient block.
- **Cryptographic Key Fingerprint (key_id):** A canonical SHA-256 digest computed over the 33-byte compressed secp256k1 public key via ECIESKeyManager.get_key_id(). This allows clients with multiple active keys or rotating credentials to disambiguate the correct key pair unambiguously.

What happens if an attacker modifies the recipient list?

The implementation enforces a strict **Fail-Closed policy**:

- The canonical serialization of both the metadata and the entire recipients array is bound directly into the AES-GCM Additional Authenticated Data (AAD) via build_aad().
- If an attacker modifies an id, alters an encrypted_key, swaps recipient ordering, injects a rogue recipient, or drops a participant, the computed GHASH tag will not match the container's tag.

- AES-GCM decryption terminates immediately, raising an IntegrityError before releasing any plaintext, thereby mitigating recipient substitution, privilege escalation, and tampering attacks.

What happens if a public key is wrong?

- **Malformed or Invalid Key Format:** Public keys are strictly validated before encryption begins. Keys must represent valid points on the secp256k1 curve. The system accepts both compressed (33 bytes) and uncompressed keys (normalizing them automatically via coincurve). Any malformed hex string or point off-curve raises a ValueError immediately.
- **Valid Key but Wrong Recipient:** If a payload is encrypted for an unintended recipient, the recipient's private key fails the ephemeral ECDH scalar multiplication and HMAC verification inside ecies.decrypt(), raising a ValueError during DEK unwrapping and aborting execution prior to AES-GCM processing.

Multi-Recipient Governance & Duplicate ID Prevention

- **Minimum Recipient Threshold:** HybridVault.encrypt_hybrid strictly enforces len(recipients) >= 2, rejecting single-recipient operations with a ValueError to ensure multi-party governance.
- **Explicit Duplicate ID Rejection:** When recipients are supplied as a list of dictionaries, duplicate identifier keys are rejected explicitly with ValueError("Duplicate recipient identifier detected...") rather than silently overwriting previous entries in the mapping.

3. Canonical Container Structure (D3 Hex Schema)

The vault container migrated from D2's Base64 format to a strictly typed, canonical JSON dictionary where all binary artifacts are represented as lowercase hexadecimal strings:

```
{
  "metadata": {
    "version": 1,
    "filename": "document.pdf",
    "aead_algorithm": "AES-256-GCM",
    "key_wrap_algorithm": "ECIES-secp256k1",
    "creation_timestamp": "2026-10-25T20:00:00+00:00"
  },
  "recipients": [
    {
      "id": "alice",
      "key_id": "8a3f... (SHA-256 fingerprint, 64 hex chars)",
      "encrypted_key": "04a1... (hex-encoded ECIES wrapped DEK)"
    },
    {
      "id": "bob",
      "key_id": "3c9b... (SHA-256 fingerprint, 64 hex chars)",
```

```
      "encrypted_key": "04b2... (hex-encoded ECIES wrapped DEK)"
    }
  ],
  "nonce": "12-byte hex string (24 characters)",
  "ciphertext": "variable-length hex string",
  "tag": "16-byte hex string (32 characters)"
}
```

4. Implementation & Team Integration

Work Division

- **Jonathan González (src/crypto/hybrid_vault.py):** Core hybrid encryption engine (HybridVault.encrypt_hybrid), recipient normalizer, multi-recipient enforcement, and public key compression bridge.
- **Jesús Paz (src/crypto/hybrid_vault.py):** Core hybrid decryption engine (decrypt_hybrid), ECIES DEK decapsulation, recipient authorization lookup, and fail-closed integrity checks.
- **Yukioayax Gabriel (src/crypto/aad_builder.py):** Canonical metadata builder, domain-separated AAD reconstruction, and strict schema validation.
- **Armando Vázquez (tests/ & Code Review):** Security audit, edge-case vulnerability testing, and unit/integration verification suite.

Integration Challenges & Peer Review Resolutions

- **Codebase Unification:** Resolved merge conflicts in hybrid_vault.py by harmonizing class-based encryption (HybridVault) with standalone functional decryption (decrypt_hybrid), maintaining complete backward compatibility.
- **Key Format Normalization:** Tampering tests generated 64/65-byte uncompressed Ethereum-style public keys via eth_keys. A transparent adapter using coincurve was embedded into encrypt_hybrid to normalize all keys to 33-byte compressed points conforming to ECIESKeyManager.
- **Exception Hierarchy Alignment:** Introduced IntegrityError(RuntimeError, ValueError) to ensure seamless handling across tests expecting either generic RuntimeError or standard ValueError.